Railway Delay Prediction and Platform Scheduling Simulation

A Minor Project Report

XGBoost prediction with PSO model selection

NDLS platform allocation and SimPy Digital Twin

Interactive dashboard and 10,000-run stress test

Prepared for academic project submission

8 November 2026

Project purpose and decision boundary

This report documents a minor-project decision-support prototype that predicts next-stop arrival delay and uses those estimates in a configurable station-platform simulation. The prediction model is the existing XGBoost model tuned with four metaheuristics. Its training and evaluation pipeline remains the same. The project adds TreeSHAP explanations, platform allocation, a SimPy discrete-event twin, an FCFS baseline, an interactive Gantt view, and a 10,000-run stress harness.

The prototype is an offline educational experiment. It is not an Indian Railways control system and does not issue movement authority, alter live timetables, or communicate with signaling equipment. The public delay extract does not establish a verified NDLS daily timetable, train/platform lengths, dwell plan, or platform assignment history. Those scheduler inputs are therefore configurable scenario assumptions. Prediction results are measured on historical records; scheduler results are measured on synthetic operating scenarios.

The main finding is mixed. The pooled predictor reports MAE 6.44 minutes, while the NDLS-only temporal holdout reports MAE 9.77 minutes. The default 36-movement scenario has no queueing under FCFS or the interval scheduler. A busy 60-movement scenario produces 906.02 total wait-minutes under both policies after a minimum-wait safeguard. The synopsis target of a 30% wait reduction is not met.

Abstract

The project builds a reusable railway delay prediction and station decision-support prototype around the yadavsaurabh/indian-railways-runs dataset. Stop-level records are paired within train journeys to predict the arrival delay reported at the immediate next stop. Features include train type, current and next station, route sequence, scheduled running and dwell time, current-stop delays, and calendar fields. Chronological partitions separate training, validation, uncertainty calibration, and final testing. Four XGBoost tuning strategies—GA, PSO, WOA, and GWO—are compared on validation RMSE. XGBoost-PSO is selected and a nominal 90% residual interval is calibrated.

The pooled final test contains 42,997 examples and yields MAE 6.44 min, RMSE 12.97 min, R^2 0.957, and 89.1% empirical coverage for the nominal 90% interval. The NDLS-only holdout contains 62 examples and yields MAE 9.77 min, RMSE 12.68 min, R^2 0.981, and 90.3% interval coverage. Native XGBoost TreeSHAP values explain uploaded predictions at original-feature level.

A deterministic weighted-interval allocator, platform length and clearance checks, SimPy queue simulation, an FCFS comparison, and a 10,000-run sensitivity harness extend the project. With 6 configured 700 m platforms, the 36-train moderate scenario gives 0 total wait in both policies. Under a 60-train busy scenario using 4 configured 700 m platforms, FCFS and the safeguarded interval plan each produce 906.02 total wait-minutes. All scenario values must be read as synthetic, not observed operations.

Executive summary

The user asked for a model beginning with NDLS and reusable across station datasets, then requested an interactive dashboard with model comparison, evaluation tables, prediction explanations, and an explanation of results. The synopsis adds station platforming, safety checks, a Digital Twin, FCFS comparison, and Monte Carlo stress testing. This implementation preserves the original prediction artifacts and adds those system-level components as a separate simulation layer.

The prediction component is complete for the current extract: seven train-history files yielded 281,252 raw stop rows and 164,297 usable one-stop-ahead examples. XGBoost-PSO had the lowest validation RMSE among the four metaheuristics. A pooled test and a distinct NDLS-only test are reported separately. TreeSHAP contributions are computed from the fitted pipeline and grouped back to the 13 original predictor columns.

Platforming is now executable and visible in Streamlit. Its configurable demo day supports 20–60 movements and 4–8 platforms. It checks train length against configured platform length and applies a minimum five-minute release clearance. A priority-aware weighted interval allocator selects compatible reservations and uses an FCFS safeguard whenever the priority plan would increase total waiting. The project demonstrates the requested workflow but does not meet every numeric target: pooled/NDLS MAE exceed 4.5 min; the busy scenario has 0% wait reduction; and a 25% secondary-platform utilization gain is not demonstrated.

Contents and reading guide

Part I establishes the project purpose, academic background, and research gap. Part II documents the dataset, feature construction, chronological evaluation, XGBoost tuning, test metrics, uncertainty calibration, and TreeSHAP implementation. Part III describes the software architecture, platform allocation, safety checks, SimPy model, FCFS baseline, and operating scenarios. Part IV covers the stress test, dashboard, reproducibility, goals, limitations, references, and appendices.

The evidence is separated into three kinds. Historical prediction scores are computed from the supplied train-history extract and its later-date holdout. Scheduler outputs are computed from generated timetable, dwell, train-length, and platform assumptions. User-interface behavior is an implemented workflow that can be rerun locally. A number from a generated scenario must not be described as a measured NDLS service statistic.

The report uses the word ‘prediction’ for next-stop delay estimates. ‘Allocation’ refers to the assignment of a simulated movement to one configured platform. ‘Simulation’ refers to event processing in SimPy with limited-capacity platform resources. ‘Stress test’ refers to randomized changes around the selected synthetic scenario. This vocabulary keeps the data-backed model results distinct from scenario-dependent planning results.

Part	Topics
I	Purpose, scope, research background, and gap
II	Dataset, features, model, evaluation, uncertainty, and explanations
III	Architecture, scheduler, safety, SimPy, baseline, and scenarios
IV	Stress test, dashboard, reproducibility, limitations, references, and appendices

Scope and synopsis traceability

The synopsis describes delay prediction, model comparison, platform allocation, length and clearance constraints, a SimPy Digital Twin, FCFS baseline, a Gantt dashboard, conflict explanations, and 10,000 randomized perturbations. The implemented prediction, model-comparison, prediction-interval, and station-scoring parts are retained. The new platforming module implements deterministic interval allocation and reports assignment, wait, utilization, throughput, runtime, and hard-check violations.

The implementation supports one selected station scenario at a time. NDLS is the starting label in the reproducible CLI and default project run. A prediction CSV can be filtered by target-station code in the dashboard, allowing another station's estimated delays to supply the scenario. Station operation configuration remains local to the scenario; station-specific infrastructure has not been loaded from an authoritative railway source.

The scope excludes live data feeds, signaling integration, dispatch authority, nationwide platform optimization, and a production safety case. The project should be presented as an offline educational decision-support prototype. Out-of-scope operations are not claimed as delivered functionality.

Problem statement

A train can arrive late at one stop and carry some portion of that delay to the next stop. Historical train-running records can support a short-horizon prediction, but a point estimate alone does not show its uncertainty or how it might affect a station decision. A station platform decision also has capacity constraints: a movement occupies one platform for a duration, and another movement needs enough release clearance before reusing it.

This project studies two connected but different questions. First, how accurately can the next reported arrival delay be estimated from information available at the current reported stop? Second, under an explicit synthetic operating scenario, how can arrivals be assigned to a small configured set of platforms while checking train length, occupancy overlap, and minimum clearance? The first question is measured against recorded delay labels. The second is tested against the scenario model's own assumptions.

The public extract does not encode a complete dispatching history or verified platform geometry. The platforming study therefore answers an algorithmic feasibility question, not a factual claim about NDLS operations. This distinction limits external validity but allows the scheduling workflow to be demonstrated and measured reproducibly.

Objectives and success criteria

The prediction objectives are to create leakage-conscious one-stop-ahead examples, compare the four metaheuristics named in the selected paper, choose the optimizer on validation RMSE, report MAE/RMSE/R^2 on later dates, and calibrate a nominal 90% interval. The dashboard exposes the workflow, station results, algorithm table, final predictions, and uploaded-row explanations.

The synopsis's system goals include at least 30% total-wait reduction against FCFS, near-zero collisions, a 25% improvement in secondary-platform use, and recomputation below 50 ms. These are targets for evaluation rather than assumed outcomes. In the tested busy scenario the wait-reduction metric is 0%; collision violations are zero under exclusive resources and explicit checks; no 25% utilization improvement is observed; and allocator time varies by machine and scenario, so a single run cannot establish the latency target.

The prediction goal MAE below 4.5 minutes is also not met by the reported temporal holdouts. An accurate report needs to show both achievements and misses. The system is useful as a completed prototype even when an empirical target is not met; the missed targets guide the next experiment, especially the need for real timetable and platform data.

Operational context

Railway delay prediction can serve passenger information, tactical planning, and operations analysis. A short-horizon estimate can support a human analyst by estimating the likely arrival at the next reported stop. A platform simulation can explore what-if scenarios when movements compete for capacity. This project combines those ideas as an offline workflow: the prediction artifact supplies delay estimates; the scenario generator adds a synthetic timetable and infrastructure configuration; the scheduler and simulator evaluate the resulting schedule.

The prototype is not a control loop. It does not receive live track-occupancy messages, model route locking, signal aspects, interlocking logic, track conflicts, passenger flows, crew or rolling-stock constraints, or human dispatcher actions. It therefore cannot authorize a train movement. A user can adjust scenario inputs and inspect consequences, but the system has no direct effect on railway service.

This boundary is essential when describing a Digital Twin. Here the term means a software simulation of a simplified station process, not a synchronized twin of a physical railway asset. The model's operational meaning depends on the fidelity of its schedule and resource configuration. This prototype gives a transparent starting structure for later validation using station-approved data.

Literature review method

The literature section draws on three related strands: systematic reviews of train-delay prediction, the user-supplied Gao XGBoost/metaheuristic paper, and foundational work on boosted trees and local explanation. The review papers provide a broad map of prediction horizons, input variables, model families, evaluation practice, data quality, and external validation. Gao et al. provide the most direct methodological comparison for XGBoost tuning by GA, PSO, WOA, and GWO.

The review does not treat published performance values as directly comparable to this project. The cited paper studies a different railway, different data, different feature set, and much larger search budget. Its dispatching-command and adjacent-train features are not available in the project's current feature matrix. Its 1.9-million-record result is background evidence, not a benchmark score for the Indian dataset.

References were checked against publisher or institutional records where available. References from the supplied older presentation that could not be verified were excluded. The review is a focused project review, not an exhaustive systematic search.

Review paper on data-driven train delay prediction

Tiong, Ma, and Palmqvist review data-driven approaches to train-delay prediction and discuss the structure of model development, input data, methods, outputs, and evaluation. Their review emphasizes that data quality, model validity, and explainability need more attention. It also discusses the difference between long-term and short-term prediction horizons and the practical importance of identifying the information available at prediction time.

The project follows those concerns through explicit data pairing, a one-stop-ahead target, a date-based split, a separate calibration partition, and an untouched final test. It includes model-importance analysis and local TreeSHAP explanations. These practices improve auditability, though they do not replace external validation on independent routes or a larger national sample.

The review also helps interpret the current metrics. High R^2 can coexist with substantial absolute error when target delays vary widely. A low average error alone does not prove performance on unusually long delays or unseen train services. Future evaluation should include additional routes, a larger held-out time period, and delay-severity slices.

Review paper on train delay prediction approaches

Spanninger, Trivella, Büchel, and Corman survey train-delay prediction methods and classify them by modeling paradigm, prediction type, input data, and horizon. The review distinguishes data-driven models from event-driven approaches and summarizes how different approaches represent propagation and operational events. It provides context for combining a statistical predictor with an event-based simulation.

This project uses a supervised, data-driven XGBoost model for the next-stop target and a separate discrete-event simulation for station resource interactions. The combination is a practical architecture rather than a claim that the predictor itself learns platform assignment. The two modules have different assumptions and separate metrics.

The review's range of approaches also cautions against treating one algorithm as universally best. The project selects XGBoost-PSO only because it minimizes validation RMSE in this extract under the implemented search budget. That decision is local to this experiment and should be re-evaluated as route coverage and data completeness improve.

Gao et al. XGBoost metaheuristic paper

Gao, Chen, and Xu describe an XGBoost framework for train-delay prediction that incorporates dispatching commands and spatiotemporal delay-propagation information. The article compares Genetic Algorithm, Particle Swarm Optimization, Whale Optimization Algorithm, and Grey Wolf Optimizer for hyperparameter tuning. It reports 1.9 million train-operation records across 38 months and uses SHAP analysis to explain model behavior.

This project adopts the four-optimizer comparison and the XGBoost parameter search ranges as a methodological reference. It does not reproduce the paper's railway, command variables, training volume, computing environment, or full optimizer budget. The project's sample uses seven cached train histories, a compact population of six, two generations, and bounded tuning samples. The chosen method here is XGBoost-PSO; Gao et al.'s paper reports different results for its own data.

This is a methodological adaptation. The paper supports the choice to compare optimizers and use explainability, but it cannot justify claims that the same features or performance have been achieved here.

Foundational XGBoost and SHAP ideas

XGBoost is a gradient-boosted tree system that adds trees sequentially to reduce a specified loss, with regularization and engineering choices intended to improve accuracy and speed. It is suitable for tabular, nonlinear relationships such as train type, station codes, scheduled running time, and current delay. The project uses the squared-error regression objective and compares hyperparameter configurations using validation RMSE.

SHAP assigns feature contributions to a prediction under an additive explanation framework. XGBoost's native TreeSHAP routine produces a baseline value and per-feature contributions. For categorical inputs, the fitted preprocessing pipeline expands a station or train category into one-hot columns. The explanation module sums the one-hot contributions back into the original project variable so a user sees a contribution for `station_code`, rather than many encoded categories.

A local contribution describes how a feature shifts a particular prediction relative to a model baseline. It does not establish causation. In this project, positive contributions raise the predicted delay and negative contributions lower it. The displayed per-row baseline plus all contributions matches the model output within floating-point tolerance.

Synthesis of the reviewed literature

The reviewed sources support a staged project design. First define the prediction target and what information is available at that moment. Next examine data quality and leakage. Then compare methods on a consistent partition, measure more than one error statistic, and add explanation and validity checks. When a model is used to support operational choices, further evaluation needs to reflect decision consequences rather than predictive accuracy alone.

Gao et al. add a concrete XGBoost tuning recipe, including four metaheuristics and SHAP. Tiong et al. emphasize model development, data quality, explainability, and validation. Spanninger et al. place data-driven and event-driven approaches in a wider taxonomy. These sources inform separate but connected prediction and simulation components in this project.

The project's main evidence gap is not another optimizer. It is validated operational input. Without true platform reservations, train lengths, dwell histories, conflict rules, and a representative timetable, a scheduler's synthetic result cannot show actual station performance. Future work should prioritize data access, independent evaluation, and expert review of constraints before attempting more complex optimization.

Research gap and contribution

The supplied synopsis proposes a practical bridge between next-stop prediction and platform allocation. The codebase originally delivered the prediction model and dashboard, but not the scheduler, event simulation, safety checks, or Monte Carlo evaluation. This implementation adds those missing software components in a transparent offline scenario workflow.

The contribution is an integrated educational prototype: XGBoost-PSO predictions with a calibrated interval and TreeSHAP explanations; a priority-aware weighted interval allocator with a minimum-wait guardrail; platform-length and clearance verification; a SimPy comparison against FCFS; an interactive schedule display; and reproducible sensitivity results. The model, optimizer comparison, and underlying prediction scores were retained while the system-level layer was added.

The contribution is implementation and demonstration, not operational proof. Because the dataset does not verify station configuration, the scheduling evaluation has limited external validity. The report records the actual measured scenario output and the gap to the synopsis targets instead of claiming success based on design intent.

Dataset source and coverage

The source is the `yadavsaurabh/indian-railways-runs` dataset card on Hugging Face. The project's current model run reads seven cached per-train history files. Together they contain 281,252 stop-level rows and cover 2020-01-01 to 2026-09-01. Feature construction produces 164,297 consecutive reported-stop examples across the selected corridor sample.

This is not the whole public corpus and not a random national sample. The histories are train- and route-biased, and the source's web viewer may expose only a partial export. Results therefore describe this cached extract. Station metrics depend on which routes and dates are present. A previous NDLS-only archive is a separate sample and is not silently merged into these current model scores.

The source card declares GPL-3.0. Users should review the dataset terms before redistribution or publication. The data provider is independent and is not affiliated with Indian Railways. A public project submission should cite the dataset card and describe how the sample was collected.

Raw schema and target definition

The raw records describe a train stop on a service date. Fields used by the feature builder include train number, date, station code, route sequence, train type, scheduled arrival and departure, distance along the route, reported status, and arrival/departure delay. Each valid example pairs one current stop with the next stop of the same train on the same journey date.

The model predicts the next stop's reported arrival delay in minutes. The feature row uses only current-stop and scheduled information: categorical train type and station codes; sequence indices; distance to the next stop; scheduled run and dwell time; current arrival/departure delay; day of week; month; and weekend flag. Actual future arrival time and future target delay are labels, not predictors.

The dataset card includes a `platform` field, but the project does not use that field as an allocator target or validate its consistency. Platform lengths and train lengths are not treated as measured variables in the scheduler. That explicit separation avoids implying that an unverified field is a ground-truth operational plan.

Data cleaning and example construction

The feature builder parses dates and numeric fields, drops rows missing required keys, deduplicates a train/date/sequence/station combination, and orders each journey by sequence. Where reported status is available, both the current and next stop must be reported. Within each train and date, the next station and its delay are obtained by shifting the ordered records by one position.

Scheduled running time is computed from current scheduled departure to the next scheduled arrival, including a day rollover when a train crosses midnight. Scheduled dwell is computed from scheduled arrival to departure at a stop. Examples without a usable next-stop target, next station, or current departure delay are removed. Source delay values are one-sided and nonnegative; early-running targets are not modeled.

This code prevents pairing stops across different journeys and excludes actual future time fields from model features. It does not repair all source-data inconsistencies. A future data pipeline should add explicit anomaly reporting for impossible sequence jumps, duplicate service-day records, missing schedule values, and inconsistent station mileage.

Feature engineering

The 13 predictors combine three categorical columns with ten numeric columns. Categorical fields are train type, current station, and next station. Numeric fields are current sequence, next sequence, distance to next stop, scheduled run time, scheduled dwell, current departure delay, current arrival delay, weekday, month, and weekend indicator.

The fitted pipeline imputes missing categorical values with the most frequent category and applies one-hot encoding with unknown categories ignored. Numeric missing values are imputed with the median and scaled. The XGBoost estimator receives the transformed matrix, while the outer pipeline keeps the original feature names for CSV prediction inputs.

Several variables have strong physical interpretation: current delay indicates inherited lateness; scheduled run and distance provide a section-time context; and station and sequence describe route location. The features do not include weather, dispatch command, adjacent train state, rolling-stock condition, or passenger load. The missing factors limit how much the model can explain or generalize across disruption regimes.

Leakage controls

A forecasting system must not use fields that would be unknown at its forecast time. The project predicts at departure from the current reported stop, so the model excludes actual arrival and departure timestamps at the next stop and does not use the next stop's delay as an input. That future delay is used only as the target label.

Stop pairing occurs within each train and journey date. Train histories are split chronologically by date rather than randomly mixing earlier and later journeys. Separate partitions are reserved for fitting, optimizer validation, residual calibration, and final testing. The final holdout is not used to select the optimizer or its parameters.

A temporal split does not establish unseen-train or unseen-route generalization when the same train services appear across the date partitions. The current evaluation therefore tests later dates for the sampled services. A stronger external validation would hold out complete train numbers, route corridors, and dates, and would document the extract creation mechanism.

Chronological evaluation design

The engineered examples are ordered by journey date and divided into approximately 60% training, 10% validation, 10% calibration, and 20% final test. The current split counts are 79,001 training, 20,648 validation, 21,651 calibration, and 42,997 test examples. The final test period covers the latest dates in the cached extract.

The validation set supports the optimizer fitness and model choice. The calibration set produces a 90th-percentile absolute residual radius. The final test set is scored once after the selected parameters are refit on training plus validation dates. This design keeps the interval calibration and optimizer selection separate from final performance scoring.

The split is chronological but route patterns overlap across partitions. If dates are sparse or service histories are uneven, calendar proportions do not imply equal service representation. Future experiments should report counts by route, train, station, month, and delay severity for each partition and add a leave-route-out evaluation.

XGBoost model and selected parameters

The final model is a scikit-learn pipeline with a ColumnTransformer and XGBoost regressor. It uses the tree booster, squared-error objective, histogram tree method, two worker threads, and deterministic random seeds. The PSO search selected 239 estimators, learning rate 0.078626, minimum child weight 5, and maximum depth 4.

The pipeline is serialized as a joblib bundle with the fitted estimator, required feature list, station label, selected algorithm, and calibrated interval radius. The pooled artifact accepts station codes not observed during fitting through unknown-category handling. Acceptance of an unseen code does not guarantee good accuracy at an unseen station.

The original predictor is kept as the project’s center. The new scheduler consumes its output CSV and does not replace the trained model or change the score calculations. This modular boundary lets a user compare prediction performance separately from scenario-level scheduling performance.

Metaheuristic comparison

All four methods use the same bounded training and validation subsets. The compact search uses a population of six and two generations, with validation RMSE as fitness. This is a compute-adapted experiment compared with the much larger search budget in Gao et al. Each optimizer is evaluated on 2,500 validation rows, while up to 6,000 training rows are used for tuning.

Validation RMSE values were: GA 13.260 min, PSO 12.287 min, WOA 12.476 min, and GWO 13.078 min. Validation MAE values were respectively 7.471, 7.156, 7.216, and 7.344 min. PSO is selected because it has the lowest validation RMSE, the metric used as the optimizer fitness. WOA is second by RMSE.

These validation values are not final test metrics. Optimizer runtimes and scores depend on the hardware, bounded sample, and small population/generation budget. A larger repeated-seed search would be needed to estimate optimizer stability. The comparison is useful as a reproducible project experiment, not as proof that PSO is universally superior.

Optimizer	Validation RMSE min	Validation MAE min	Search seconds	Decision
XGBoost-GA	13.260	7.471	61.1	
XGBoost-PSO	12.287	7.156	50.2	Selected
XGBoost-WOA	12.476	7.216	32.4	
XGBoost-GWO	13.078	7.344	53.1	

Final prediction results

The pooled model scores 42,997 untouched test examples with MAE 6.438 min, RMSE 12.965 min, and R^2 0.9567. The nominal 90% absolute-residual interval has a calibrated radius of 13.08 min and empirical test coverage of 89.1%. The test spans many target stations in the bounded sample.

The NDLS-specific model has 62 future-date test examples. Its MAE is 9.772 min, RMSE 12.683 min, and R^2 0.9807; the nominal 90% interval covers 90.3% of these examples. The NDLS calibrated radius is 20.76 min. The small NDLS test size makes this score uncertain.

The synopsis's desired MAE below 4.5 min is not met by either pooled or NDLS-specific score. The pooled MAE summarizes a mix of stations and has a larger denominator; it should not be used to claim the NDLS-specific model has the same accuracy. Future work needs more NDLS train histories and explicit external evaluation.

Model	N	MAE min	RMSE min	R^2	90% coverage
Pooled	42997	6.438	12.965	0.9567	89.1%
NDLS only	62	9.772	12.683	0.9807	90.3%

Station-level results

The pooled model is also scored by target station when the final holdout contains at least 50 examples for that station. Example results include AGC with 346 examples, MAE 6.045 min, RMSE 6.623 min, and R^2 0.981; GWL with 281 examples, MAE 5.136 min, RMSE 7.077 min, and R^2 0.985. The NDLS-specific test is scored separately and is not conflated with pooled station performance.

Station-level metrics help identify where pooled scores vary, but a high R^2 can reflect wide target variation and should be read beside MAE and RMSE. A small sample can produce unstable values. Stations with fewer than 50 examples are excluded from the station-score table to avoid overinterpreting very small groups.

The dashboard provides a station selector, sample size, MAE, RMSE, R^2, and a ranked bar chart. Users can inspect the full available station table. It supports exploratory analysis of the current split, not certification of model performance for all Indian stations.

Uncertainty calibration

The system estimates a nominal 90% prediction interval by computing absolute residuals on a separate calibration partition. The 90th percentile of those residuals is used as a symmetric radius around each prediction. For delay outputs, the lower interval endpoint is clipped to zero because the source labels do not include early running.

The pooled interval radius is about 13.08 minutes and achieves 89.1% coverage on the final pooled test. NDLS uses its own radius of about 20.76 minutes and achieves 90.3% coverage on 62 holdout examples. Calibration is empirical under the sampled distribution; the interval may not retain nominal coverage at an unseen station, during unusual disruptions, or under data drift.

The dashboard displays point estimates and lower/upper ranges. The interval is not a guarantee that a future delay will fall within the band. It is a calibrated uncertainty summary whose practical quality depends on representative calibration examples and temporal stability.

TreeSHAP explainability

The project uses XGBoost's native TreeSHAP contribution calculation on the fitted pipeline. It transforms the original feature row with the same fitted preprocessing, calculates contribution values over encoded tree inputs, and sums one-hot category contributions back into the original 13 feature names. For each prediction, the sum of baseline value and feature contributions reconstructs the XGBoost output within floating-point tolerance.

A five-row smoke check using NDLS examples found a maximum additive reconstruction difference of approximately 0.000042 minutes. This is a numerical consistency check on the explanation implementation, not a statistical model evaluation. The dashboard presents mean absolute contributions for uploaded rows and the signed feature contributions for an individual case.

Contribution direction is local: a positive value moves the estimate above the baseline and a negative value moves it below. TreeSHAP is not causal attribution. The global permutation-importance analysis recorded in the training evaluation is a separate feature-ranking method; both views can be useful, but their values should not be compared as if they shared units or definitions.

Software architecture

The project has four logical layers. Data and feature code loads stop histories and builds one-stop-ahead examples. The training layer tunes and fits XGBoost, calibrates intervals, scores stations, and serializes artifacts. The dashboard layer presents evaluation, station comparisons, uploaded predictions, TreeSHAP explanations, and platform scenarios. The platforming module creates a synthetic operating day, applies interval allocation, and simulates queue behavior in SimPy.

The prediction bundle is loaded by the dashboard for interactive scoring. The user can download its prediction table and upload that file to the platform tab. The platform scenario samples estimated delays from the file, then adds explicit synthetic timetable slots, dwell durations, train lengths, and priorities. Outputs include the optimized schedule CSV, FCFS schedule CSV, and a JSON metrics record when the CLI is used.

This separation maintains the existing prediction pipeline while making the additional system layer testable and explainable. It also makes the data boundary visible: the model output is data-backed, while operating scenario fields are generated or configured.

Prediction-to-scheduling interface

The dashboard’s existing Make predictions tab accepts an engineered feature CSV and creates predicted delay, lower interval, and upper interval columns. It retains the human-readable summary, plotted estimates, uncertainty whiskers, and detailed table. The platform tab accepts the exported prediction-results CSV and requires a `predicted_delay_min` column. If the rows contain a next-station code, the user can select a target station before scenario generation.

The scenario samples those predicted delays and adds the scheduled arrival slot to estimate an arrival time. Its 90% prediction range is retained for review, but the present allocator schedules using the point estimate. A more advanced extension could use the interval directly to reserve robust buffers or compare low/base/high arrival scenarios.

The interface makes no claim that a user-uploaded file is an authoritative real-time train feed. The input file should come from the project prediction workflow and its station selection should match the intended demo. The UI labels timetable, dwell, train length, priority, and platform settings as configurable synthetic assumptions.

Platform allocation model

A movement is represented by a unique movement ID, train identifier, estimated arrival time, dwell duration, train length, and priority level. Its nominal occupancy interval begins at estimated arrival and ends after dwell. A platform is available only when the train length fits and the previous movement has completed dwell plus the required clearance. The default configuration uses six 700 m demo platforms and a five-minute clearance; the dashboard can configure four to eight platforms.

The scheduler assigns an interval weight based on priority and train length, applies weighted interval scheduling on candidate platform reservations, and reconsiders excluded movements in priority order. When the resulting plan has greater total wait than a feasible earliest-available FCFS schedule, the FCFS safeguard is selected. That guardrail prevents the chosen plan from increasing total wait in the tested case, though it can reduce or remove the priority-based advantage.

The method is deterministic for a fixed scenario and seed. It is a small educational allocator rather than a complete station-platform optimization model. It does not represent routing conflicts, platform groups, passenger access, route locking, or train-specific track compatibility.

Weighted interval scheduling

For one platform, weighted interval scheduling sorts candidate intervals by end time and calculates the last compatible predecessor for each interval. Dynamic programming compares the objective value from taking an interval with the value from skipping it. Backtracking returns the compatible set with the largest total weight. Intervals are compatible when the earlier interval ends at least the minimum clearance before the later one begins.

For multiple platforms, this prototype first places candidate intervals on a currently less-loaded compatible platform, applies the single-platform weighted interval selection to each candidate set, and reassigns excluded movements to the earliest feasible gap. Priority changes interval weight and reinsertion order. The final schedule is checked again for platform length and clearance.

Because this assignment decomposition is heuristic, it may not find the global optimum across all platform assignments. The FCFS safeguard compares total wait after constructing a feasible earliest-available plan. The implementation chooses the lower-wait plan, so the weighted interval component is a candidate planner rather than a mathematical guarantee of global optimality.

Priority handling and reservation changes

The scenario assigns integer priorities 1–3, with a higher number representing a stronger service priority. Weighted interval selection favors higher-priority movements when candidate intervals conflict. Excluded movements are reconsidered in descending priority order, with arrival time and movement ID resolving ties. When a movement is moved, it is placed at the earliest feasible slot on a length-compatible platform.

Reservation changes apply before simulated service starts. The simulator never interrupts a train already occupying a platform. This is a simple form of preemption at the planning stage: a higher-weight candidate can displace a lower-weight planned reservation, which is then reinserted. The system does not model operational command authority or the practical restrictions around changing a published platform assignment.

The minimum-wait safeguard can choose FCFS when a priority plan would increase total wait. In that case, the output explains why the safeguard was used. This is intentional transparency: the simulator reports both priority effects and total-wait consequences rather than assuming that a priority rule always improves system performance.

Safety and feasibility checks

Every assigned movement must satisfy train length less than or equal to the configured platform length. Each platform has a single-capacity resource, so it serves one movement at a time. The next movement must start no earlier than the prior scheduled end plus the minimum clearance. The default clearance is five minutes and the dashboard slider cannot go below five minutes.

The allocation result records assigned and unassigned movements, length violations, clearance violations, and allocator time. An overlength movement is left unassigned rather than forced onto a short platform. The schedule is checked after selection, including after the FCFS safeguard. The SimPy resource also prevents simultaneous platform occupation in the queue replay.

A zero violation count describes checks in the simulation’s simplified resource model. It does not certify real operational safety. Real railway safety depends on track layout, interlocking, signal logic, route conflicts, train braking and separation, platform geometry, local rules, and approved dispatch procedures that are outside the dataset and current simulator.

SimPy discrete-event twin

The simulator creates one limited-capacity SimPy resource per configured platform. Each movement process waits until its scheduled request time, requests the selected platform, records service start, occupies the platform for the scenario dwell duration, then holds the resource through the clearance period. SimPy advances from event to event rather than stepping through every simulated minute.

For each plan, the simulator records total, mean, and 95th-percentile wait, utilization, collision count, served movements, throughput, and per-train wait. Utilization divides occupied dwell time by the scenario horizon and the number of platforms. Per-platform utilization is also reported so the dashboard can show how activity is distributed.

The ‘Digital Twin’ in this project is an offline process simulation. It is not synchronized with a live station or physical asset. Its event rules are deliberately simple and editable so students can inspect how platform capacity, dwell, clearance, and arrival bunching affect waits.

First come first served baseline

The FCFS baseline sorts movements by estimated arrival and assigns each to the length-compatible platform that becomes available earliest. It begins service at the later of the arrival estimate and platform availability. After dwell, that platform remains unavailable for the clearance period. This creates a transparent resource-capacity benchmark using the same scenario arrivals, dwell times, train lengths, platform count, and clearance as the interval plan.

FCFS is a strong baseline for this simplified setting because all platforms are treated as equivalent except for configured length. It is not a deliberately weak schedule. Therefore the project should not expect a large improvement merely from adding a more complicated priority rule. A richer model would need platform-group restrictions, route conflict matrices, priority service definitions, and predicted consequences for downstream trains.

The comparison reports the percent change in total wait relative to FCFS. If FCFS total wait is zero, percentage change is defined as zero because the baseline offers no positive delay to reduce. In a congested case, the scheduler’s safeguard chooses FCFS when a priority candidate plan has higher total wait.

Scenario generation and assumptions

The generator supports 20–60 movements and three synthetic demand windows: light, moderate, and busy. It samples model delay estimates when the user supplies a prediction CSV; otherwise it uses seeded synthetic delays. Timetable arrival slots are generated within a six-, ten-, or eighteen-hour operating window depending on demand. Dwell, length, and priority are sampled from fixed demo ranges.

The default reproducible run uses seed 42, 36 movements, six platforms, 700 m platform lengths, moderate demand, and five-minute clearance. The congested comparison uses seed 7, 60 movements, four platforms, 700 m lengths, busy demand, and the same clearance. All these figures are experiment settings, not observations of NDLS.

The scenario’s role is to exercise the allocation and event simulation. Users can export the generated schedules and adjust the dashboard controls. For an operational study, synthetic timetable and length values must be replaced by station-approved schedules, infrastructure records, and validated dwell distributions.

Default moderate scenario results

The default scenario has 36 movements and 6 configured platforms, all 700 m long, with a five-minute clearance. It assigns 36 of 36 movements and records 0 length violations and 0 clearance violations.

The FCFS and selected interval plans each produce 0.0 total wait-minutes. Aggregate platform utilization is 21.24%, and the per-platform utilization ranges from 2.58% to 53.38%. Since there is no queueing, the scenario does not show a wait reduction or a utilization improvement.

This is a low-conflict operating case under the chosen assumptions. Its main value is checking that all movements can be assigned and that the schedule and event model agree on a sparse day. It should not be used to infer that NDLS has low platform utilization or no conflicts in real service.

Measure	Value
Movements / platforms	36 / 6
FCFS / selected wait	0.0 / 0.0 min
Aggregate utilization	21.24%
Length / clearance violations	0 / 0

Busy scenario results

The busy scenario uses 60 movements and 4 configured platforms. The generator assigns all 60 movements with no length or clearance violations. Its aggregate utilization is 72.49% under the simulated horizon.

FCFS total wait is 906.02 minutes. The initial priority interval plan would have had a higher wait, so the FCFS safeguard is selected; the final interval/safeguarded plan also has 906.02 total wait-minutes. The resulting reduction is 0.0%. The configured platforms have individual utilizations of P1: 69.1%, P2: 75.2%, P3: 70.4%, P4: 75.2%.

The result shows the value of retaining an FCFS benchmark and comparing the actual objective. Under these assumptions, the weighted priority rule does not reduce aggregate waiting. No claim of 30% improvement or better secondary-platform use is supported.

Measure	FCFS	Selected plan
Total wait min	906.02	906.02
Aggregate utilization	72.49%	72.49%
Wait reduction	—	0.0%
Safety violations	0	0

Monte Carlo stress method

The stress harness runs 10,000 seeded perturbations of one selected scenario. Each run adds a normal arrival-time shock with mean zero and standard deviation five minutes, multiplies dwell duration by a uniform factor between 0.85 and 1.35, and, when more than four platforms are configured, assumes a 2% chance that one platform is unavailable. The resulting movement stream is replayed through length-compatible single-capacity platform queues.

The output reports the distribution of total wait, runs with static conflict risk under the nominal platform assignment, resource collisions after queue recovery, unassigned movements, and runtime. It also reports median, 95th, 99th percentile, and maximum waits. The outage probability and perturbation ranges are sensitivity choices and are not estimated from empirical railway data.

The zero post-recovery resource-collision count is guaranteed by the exclusive resource queues in the simulation. The separate static-conflict-risk count is the more informative stress measure: it identifies perturbed runs where the nominal reservation would overlap and a queue or reassignment is needed. Neither metric is a safety certification.

Monte Carlo results for the default scenario

In the 10,000-run default sensitivity test, mean total wait is 0.184 minutes and median wait is 0.0. The 95th percentile is 0.0 minutes; the 99th percentile is 6.61 minutes; maximum wait is 21.05 minutes.

Static conflict risk before queue recovery occurs in 1,021 of 10,000 runs (10.21%). The average number of adjacent risk pairs is 0.1057 per run. The queue replay has zero resource collisions and no unassigned movement in this setup.

The stress replay's average runtime is 0.393 ms per perturbation and its 95th percentile is 0.699 ms. This is the fast sensitivity replay, not the complete allocator runtime. The scenario is reproducible with seed 2068 and the CLI.

Stress measure	Result
Runs	10,000
Mean total wait	0.184 min
P99 total wait	6.61 min
Static conflict-risk runs	1,021
Post-queue resource violations	0
Average replay runtime	0.393 ms

Dashboard workflow

The Streamlit dashboard retains the prediction tabs and adds a Platform twin tab. Existing sections show workflow steps, final model results, metaheuristic comparison, station scores, row-level prediction audit, new prediction CSV scoring, and data/metric definitions. The prediction view keeps the table and adds summary cards, a point-and-range chart, and TreeSHAP contribution summaries.

The platform tab lets the user choose movement count, platform count, demo platform length, minimum clearance, demand pattern, seed, and optional prediction-results CSV. When the uploaded rows include next-station codes, the user can filter to the intended target. The interface then shows FCFS and selected-plan waits, utilization, safety counts, per-platform utilization, an interactive Gantt chart, allocation explanations, and schedule download.

The stress test runs only when the user presses its button. It exposes the assumption ranges and writes a detailed result object to the screen. The tab labels scenario inputs as synthetic/configurable. This keeps the prediction workflow intact while showing the new system-level experiment in one place.

Reproducing the project

From PowerShell, activate the project virtual environment, install requirements, and launch Streamlit from the repository root. The additional dependency is SimPy. To run the dashboard, use `python -m streamlit run src/dashboard.py`. The dashboard uses the existing trained model bundle and evaluation JSON under `output/`.

To create an offline scenario and save its results, set `PYTHONPATH` to `src` and run `python src/run_platform_simulation.py --station NDLS --movements 36 --platforms 6 --demand moderate --seed 42`. The command writes platform twin JSON plus FCFS and optimized schedule CSV files in `output/`. A CSV exported from Make predictions can be passed with `--predictions path\to\predictions.csv`.

To repeat the busy case, use 60 movements, 4 platforms, busy demand, and seed 7, with an output directory such as `output/ndls_busy_scenario`. Fixed seeds make synthetic input draws reproducible. The machine-specific allocation runtime can vary and should be measured again on the target laptop.

Code and output map

`src/railway_delay/features.py` builds the next-stop examples. `model.py` trains and evaluates the XGBoost pipeline, while `xgb_optimization.py` implements the four compact metaheuristic searches. `predict.py` scores engineered CSVs. `explainability.py` returns TreeSHAP contributions grouped to original features. `platforming.py` generates demo inputs, allocates platforms, runs SimPy comparisons, and executes the stress replay.

`src/dashboard.py` exposes the model workflow, station metrics, prediction upload, TreeSHAP explanation, and interactive platform twin. `src/run_platform_simulation.py` saves reproducible scenario results. `requirements.txt` declares the project dependencies, including SimPy. The training artifacts and model evaluation remain in `output/`.

Current outputs include `evaluation.json`, the pooled and NDLS model bundles, the optimizer comparison table, row-level holdout predictions, default and busy scenario metric JSON files, and the FCFS and selected schedule CSVs. These are the evidence artifacts used in this report.

Evaluation against synopsis targets

Prediction accuracy target: the synopsis names MAE below 4.5 minutes. The pooled holdout MAE is 6.44 minutes and the NDLS-only MAE is 9.77 minutes, so neither meets the target. The pooled nominal 90% interval coverage is 89.1%; NDLS coverage is 90.3% on 62 test rows.

Scheduling target: at least 30% total-wait reduction over FCFS. The default scenario has no FCFS wait to reduce. The 60-movement busy scenario records 906.02 min under both FCFS and the final safeguarded plan, so reduction is 0%. Safety target: the model reports zero simulated length/clearance violations and zero resource collisions after queue recovery; the latter follows from the resource design and is not independent railway safety evidence.

Secondary-platform utilization target: 25% improvement is not demonstrated in these two scenarios. Runtime target: the saved allocator runtime is scenario- and machine-dependent and one measurement cannot establish a sub-50 ms guarantee. The code reports it so the user can repeat measurements under the chosen movement count and platform configuration.

Synopsis goal	Measured result	Status
Prediction MAE <4.5 min	Pooled 6.44; NDLS 9.77	Not met
Wait reduction ≥30%	Busy scenario 0.0%	Not met
Near-zero simulated conflicts	0 modeled violations in scenarios	Met within simulation rules
Secondary utilization +25%	No such gain observed	Not met
Recompute <50 ms	Varies by run; busy saved run >50 ms	Not established

Limitations

The prediction sample contains seven train histories and is biased toward those routes and dates. The temporal holdout shares services with earlier partitions and does not establish unseen-train performance. NDLS has only 62 examples in the station-specific test. The data contains clipped nonnegative delay labels and lacks the dispatching-command features in Gao et al.

The scheduler uses synthetic timetable slots, dwell, priority, train length, and platform lengths. It simplifies station resources to independent single-capacity platforms and does not model tracks, route conflicts, occupancy circuits, platform groups, passenger access, signaling, cancellations, overtakes, or operational dispatch rules. The stress distributions are assumption-driven, and queue serialization structurally prevents resource collisions.

Prediction errors and synthetic schedule results cannot be merged into a single claim of improved real railway performance. Platform allocation should not be used for live decisions. Improvement requires better source coverage, station-approved configuration, external validation, and review by railway operations and safety experts.

Discussion

The strongest implemented result is a cohesive, inspectable project workflow. The predictor has a clean separation between fitting, validation, calibration, and test. The optimizer comparison uses identical subsets and a clear selection metric. The prediction interval and SHAP view add uncertainty and explanations. The system extension consumes model outputs without changing the original trained model.

The scheduler evaluation shows why a transparent baseline matters. A priority-aware interval plan may satisfy local constraints while still increasing total wait. The added safeguard selects the lower-wait FCFS plan, preventing regression in the tested objective but not creating a wait reduction. In the moderate case, there is no queue at all. These results suggest the current scenario needs richer operational constraints and stronger optimization methods before the synopsis's improvement target can be tested meaningfully.

The project now demonstrates the requested sequence of prediction, allocation, event simulation, explanation, and stress testing. Its next research step is not to claim success from synthetic runs; it is to replace scenario assumptions with validated station data and evaluate both safety feasibility and passenger/service consequences.

Conclusion

The project retains the XGBoost-PSO next-stop delay model and adds the missing platforming and simulation workflow: weighted interval selection, priority-aware reservation changes, length and clearance checks, SimPy event processing, FCFS comparison, TreeSHAP explanations, interactive Gantt chart, and a 10,000-run sensitivity test. The implementation is reproducible from the repository and the default scenario outputs are saved in JSON and CSV form.

The pooled predictor reaches MAE 6.44 minutes and the NDLS-specific predictor reaches 9.77 minutes on their chronological test sets. The default scenario assigns all 36 movements with no wait. Under the busy 60-movement scenario, the FCFS safeguard returns a schedule equal to FCFS at 906.02 total wait-minutes. The synopsis's accuracy and 30% wait-reduction targets are not achieved in the measured runs.

The result is an educational prototype, not a live Digital Twin or an approved railway tool. Its next improvement depends on validated operating data, not stronger claims about synthetic results.

Future work

First, expand the prediction sample across routes and train services, validate the extraction process, and repeat model scoring on complete unseen services and later dates. Evaluate high-delay tails and report uncertainty coverage by station and severity. Add weather, dispatch commands, and adjacent-train features only when these fields can be aligned and verified.

Second, obtain an authoritative station timetable, platform layout and lengths, rolling-stock lengths, dwell distributions, route compatibility, and operational clearance rules. Replace random demo values with those sources. Ask railway operations experts to review the model assumptions and distinguish planning constraints from safety-critical constraints.

Third, compare the current WIS/FCFS safeguard with stronger multi-platform scheduling methods using identical scenarios and explicit objectives. Repeat seeds and report distributions for wait, utilization, throughput, unassigned movements, and run time. Add robust scheduling that uses the prediction interval rather than only the point estimate, and maintain a human approval boundary.

References

[1] Gao, T., Chen, J., & Xu, H. (2024). Data-driven train delay prediction incorporating dispatching commands: An XGBoost-metaheuristic framework. IET Intelligent Transport Systems, 18(10), 1777–1796. https://doi.org/10.1049/itr2.12461

[2] Tiong, K. Y., Ma, Z., & Palmqvist, C. W. (2023). A review of data-driven approaches to predict train delays. Transportation Research Part C: Emerging Technologies, 148, 104027. https://doi.org/10.1016/j.trc.2023.104027

[3] Spanninger, T., Trivella, A., Büchel, B., & Corman, F. (2022). A review of train delay prediction approaches. Journal of Rail Transport Planning & Management, 22, 100312. https://doi.org/10.1016/j.jrtpm.2022.100312

[4] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://doi.org/10.1145/2939672.2939785

[5] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30.

[6] SimPy contributors. SimPy 4 documentation: shared resources and discrete-event processes. https://simpy.readthedocs.io/en/stable/

[7] yadavsaurabh. Indian Railways Runs dataset. Hugging Face Datasets. https://huggingface.co/datasets/yadavsaurabh/indian-railways-runs (dataset card declares GPL-3.0).

Appendix A Metric definitions

MAE is the mean absolute difference between target and prediction and is easy to interpret in minutes. RMSE is the square root of mean squared error and penalizes large misses more strongly. R^2 compares squared prediction error with the variance around the mean baseline. R^2 may be high even when absolute errors remain meaningful, especially when target values span a wide range.

Interval coverage is the fraction of final test labels falling inside the stated lower and upper prediction bounds. Total wait is the sum of time each simulated movement waits after its estimated arrival until service starts. Platform utilization is occupied dwell time divided by the simulated horizon and available platform capacity. Throughput is completed movements divided by the simulated horizon in hours.

A collision/resource violation in the simulation means overlapping use of the same configured resource or failure of a modeled length/clearance constraint. It does not stand for all railway safety hazards. Static conflict risk is counted before queue recovery when a perturbation would make nominal reservations overlap. Allocator runtime and stress-replay runtime are separate measurements.

Appendix B Prediction feature glossary

`train_type` identifies the train category. `station_code` is the current stop, while `next_station_code` is the immediate next reported stop. `seq` and `next_seq` locate the current and target events in the route order. `distance_to_next_km` is the section distance derived from route mileage.

`scheduled_run_min` measures the scheduled time from current departure to next arrival. `scheduled_dwell_min` represents scheduled time at the current stop. `departure_delay_min` and `arrival_delay_min` are current-stop reported delays known at prediction time. `day_of_week`, `month`, and `is_weekend` encode the calendar context.

The target is `target_delay_min`, the reported arrival delay at the next stop. It is not included in the feature matrix. Prediction CSVs must contain all model-required fields; the dashboard reports missing columns before scoring. The output adds point estimate and calibrated lower and upper bounds.

Appendix C Platform scenario fields

Each synthetic row has a unique movement ID, a train number or demo identifier, source and target station codes when available, scheduled arrival slot, model-predicted delay, lower and upper prediction range, estimated arrival, train length, dwell, priority, and clearance. Platform IDs and their configured lengths are provided separately in the scenario controls.

The scheduler writes a platform ID, scheduled service start and end, and a plain-language allocation explanation. If a movement is too long for every configured platform, it remains unassigned. If a reservation is delayed, its explanation identifies the interval conflict or minimum-wait safeguard.

Synthetic scenario rows should remain labeled as demo data when exported. A user may use prediction results from actual historical examples, but the generated timetable, dwell, length, and platform metadata remain assumptions unless replaced by verified inputs.

Appendix D Reproducibility record

The prediction artifact records algorithm XGBoost-PSO, selected PSO parameters, model feature names, and the calibrated interval. The pooled test and station results are serialized in `output/evaluation.json`; holdout rows are in `output/test_predictions.csv`; optimizer comparison values are in `output/algorithm_comparison.csv`.

The default platform scenario uses seed 42 and a stress seed of 2068. Its evaluation JSON is `output/platform_twin_evaluation.json`; schedule exports are `output/fcfs_platform_schedule.csv` and `output/optimized_platform_schedule.csv`. The busy case uses seed 7 and is saved under `output/ndls_busy_scenario/`.

The runtime environment is Python with the dependencies in requirements.txt. Installing the environment and rerunning the documented commands should recreate the feature artifacts and simulation outputs. The training process is computationally heavier than running the saved model or scenario. Hardware-dependent durations should be recorded with each repeated experiment.